

DES4069 — Agregado, Correlações e Regressão — SRAG 2026

Replicação da análise de 2024 com dados de 2026, para checar se o padrão se repete

Grupo — DES4069

30/08/2026

Contents

0.1	Por que agregar por UF (unidade de análise)	2
1	Importação do banco bruto — via API dados.gov.br	2
2	Idade corrigida (converte dias/meses para anos em vez de descartar)	4
3	Funções auxiliares de proporção	4
4	Agregação: variáveis principais (n_casos, idade, desfechos)	5
5	Agregação de todas as proporções (fatores de risco, vacina, UTI)	5
6	Conferência	6
7	Passo 1 — Correlação de cada candidata com a resposta	9
8	Passo 2 — Modelo preliminar com as 5 maiores correlações entre as candidatas clinicamente diretas (1ª rodada)	10
9	Passo 3 — Diagnóstico de multicolinearidade (VIF e correlação entre regressores)	12
10	Passo 4 — Reespecificação: removendo a redundância	14
11	Passo 5 — Triagem das comorbidades adicionais (2ª rodada)	15
12	Passo 6 — a nova variável torna alguma do modelo intermediário desnecessária?	16
12.1	Resumo do processo de seleção	17

13 Ajuste e Diagnóstico do Modelo Final	18
13.1 Tabela ANOVA	19
13.2 Intervalos de confiança dos coeficientes	19
13.3 Diagnóstico de multicolinearidade (VIF e índice de condição)	19
13.4 Comparação com o modelo reduzido (teste F parcial da última variável adicionada)	19
13.5 Gráficos de diagnóstico padrão dos resíduos	20
13.6 Resíduos studentizados internos e R-student	21
13.7 Resíduos vs. cada regressor	22
13.8 Estatística PRESS	23
13.9 Alavancagem e Distância de Cook	23
14 Equação do Modelo Final	25

15 Comparação final com 2024 25

Réplica exata da metodologia usada em 2024 (`DES4069_Seminario.Rmd`), agora com os dados de 2026. O objetivo não é reencontrar o mesmo modelo final — é rodar o mesmo processo de seleção de variáveis e ver **quais** variáveis sobrevivem desta vez. Todo texto interpretativo que dependa do resultado está marcado com `[PREENCHER]` — escreva a leitura depois de rodar, olhando os números que o próprio código produziu, não antes.

0.1 Por que agregar por UF (unidade de análise)

Mesmo argumento de 2024: o modelo de regressão linear clássico assume erros independentes, $\varepsilon_i \sim N(0, \sigma^2)$ i.i.d. (Cap. 2, MPV). Os microdados do SIVEP-Gripe vêm por **caso individual** — usar cada paciente como observação violaria essa suposição, já que pacientes da mesma UF compartilham rede hospitalar, cobertura vacinal e perfil socioeconômico regional. Agregar por UF ($n = 27$) torna as observações razoavelmente independentes, ao custo de uma amostra pequena — por isso o cuidado extra na seleção de variáveis mais adiante.

1 Importação do banco bruto — via API dados.gov.br

Diferente de 2024 (CSV baixado manualmente), esta versão baixa o CSV nacional de 2026 direto da API do dados.gov.br — a mesma que já usamos no `sragUnificado_V2.R` para o fallback do dashboard da 15ª RS. Todas as variáveis usadas nesta análise (`SG_UF_NOT`, `EVOLUCAO`, `NU_IDADE_N`, `TP_IDADE` e as comorbidades) já foram confirmadas como presentes no CSV público — só campos de identificação pessoal (nome, CPF, endereço, bairro) ficam de fora, e nenhum deles é usado aqui.

Requer `DADOS_GOV_TOKEN` configurado em `~/.Renviron` (mesmo token do dashboard). O arquivo baixado fica em cache local — `reknit` não rebaixa.

```
rm(list = ls(all = TRUE)) # limpa o ambiente antes de rodar do zero

ID_CONJUNTO_SRAG <- "39a4995f-4a6e-440f-8c8f-b00c81fae0d0" # SRAG 2019 a 2026
ANO_ANALISE_REG <- 2026

obter_token_dados_gov <- function() {
```

```

token <- Sys.getenv("DADOS_GOV_TOKEN")
if (token == "") {
  stop(
    "DADOS_GOV_TOKEN não configurado. Adicione a linha ",
    "DADOS_GOV_TOKEN=seu_token em ~/.Renviron e rode readRenviron('~/.Renviron')."
  )
}
token
}

localizar_recurso_ano <- function(ano, formato = "CSV") {
  token <- obter_token_dados_gov()
  resp <- httr::GET(
    paste0("https://dados.gov.br/dados/api/publico/conjuntos-dados/", ID_CONJUNTO_SRAG),
    httr::add_headers(`chave-api-dados-abertos` = token)
  )
  if (httr::status_code(resp) != 200) {
    stop("Falha ao consultar a API (status ", httr::status_code(resp), ").")
  }
  recursos <- httr::content(resp, as = "parsed")$recursos
  link <- NULL
  for (r in recursos) {
    formato_ok <- !is.null(r$formato) && toupper(r$formato) == toupper(formato)
    titulo_ok <- !is.null(r$titulo) && grepl(as.character(ano), r$titulo)
    if (formato_ok && titulo_ok) link <- r$link
  }
  if (is.null(link)) stop("Recurso ", formato, " de ", ano, " não encontrado na API.")
  link
}

diretorio_cache <- "cache_dados_regressao"
if (!dir.exists(diretorio_cache)) dir.create(diretorio_cache)
caminho_bruto <- file.path(diretorio_cache, paste0("SRAG_API_", ANO_ANALISE_REG, ".csv"))

if (!file.exists(caminho_bruto)) {
  link <- localizar_recurso_ano(ANO_ANALISE_REG, "CSV")
  message("Baixando via API dados.gov.br: ", link)
  download.file(link, destfile = caminho_bruto, mode = "wb")
} else {
  message("Usando arquivo em cache: ", caminho_bruto)
}

# [Não verificado nos dados] mesma convenção usada no sragUnificado_V2.R:
# separador ";", encoding latin1, tudo lido como texto (col_types "c") para
# evitar erro de parsing - comparações abaixo usam strings ("1", "2", ...)
dados_brutos <- readr::read_delim(
  caminho_bruto, delim = ";",
  locale = readr::locale(encoding = "latin1"),
  col_types = readr::cols(.default = "c"),
  progress = FALSE
)

ufs_validas <- c("AC", "AL", "AP", "AM", "BA", "CE", "DF", "ES", "GO", "MA", "MT", "MS",

```

```

      "MG", "PA", "PB", "PR", "PE", "PI", "RJ", "RN", "RS", "RO", "RR",
      "SC", "SP", "SE", "TO")
dados_brutos <- dados_brutos[dados_brutos$SG_UF_NOT %in% ufs_validas, ]

```

2 Idade corrigida (converte dias/meses para anos em vez de descartar)

Mesma correção de 2024: NU_IDADE_N só é “idade em anos” quando TP_IDADE == 3; para TP_IDADE == 1 está em dias, para == 2 em meses. Converter em vez de descartar evita jogar fora justamente os casos mais jovens (que respondem por boa parte do SRAG).

$$\text{idade_anos} = \begin{cases} \text{NU_IDADE_N}/365,25 & \text{se } TP_IDADE = 1 \text{ (dias)} \\ \text{NU_IDADE_N}/12 & \text{se } TP_IDADE = 2 \text{ (meses)} \\ \text{NU_IDADE_N} & \text{se } TP_IDADE = 3 \text{ (anos)} \end{cases}$$

```

tp_idade_num <- as.numeric(dados_brutos$TP_IDADE)
nu_idade_n_num <- as.numeric(dados_brutos$NU_IDADE_N)

dados_brutos$idade_anos_corrigida <- ifelse(
  tp_idade_num == 1, nu_idade_n_num / 365.25, # dias -> anos
  ifelse(tp_idade_num == 2, nu_idade_n_num / 12, # meses -> anos
    nu_idade_n_num) # já em anos (TP_IDADE == 3)
)

# [PREENCHER] compare com 2024 (39,4 anos sem correção vs. 29,6 com
# correção) - o desvio se repete em 2026?
mean(nu_idade_n_num[tp_idade_num == 3], na.rm = TRUE)

```

```
## [1] 34.91738
```

```
mean(dados_brutos$idade_anos_corrigida, na.rm = TRUE)
```

```
## [1] 25.16817
```

3 Funções auxiliares de proporção

Mesma lógica de 2024: proporção de “Sim” entre respostas válidas (Sim/Não), excluindo Ignorado e em branco do denominador — evita viés sistemático de UFs com mais campos não preenchidos aparecerem com proporções artificialmente mais baixas.

```

prop_sim <- function(x, validos = c("1", "2")) {
  x <- as.character(x)
  denom <- sum(x %in% validos, na.rm = TRUE)
  if (denom == 0) return(NA)
  sum(x == "1", na.rm = TRUE) / denom
}

ufs <- unique(dados_brutos$SG_UF_NOT)

```

4 Agregação: variáveis principais (n_casos, idade, desfechos)

Mesma variável resposta de 2024 — `taxa_letalidade`, a fração de casos com desfecho conhecido que terminou em óbito por SRAG:

$$\text{taxa_letalidade}_{UF} = \frac{\text{óbitos}_{UF}}{\text{óbitos}_{UF} + \text{curas}_{UF} + \text{óbitos por outra causa}_{UF}}$$

```
agregado <- data.frame(UF = ufs)
agregado$n_casos <- sapply(ufs, function(u) sum(dados_brutos$SG_UF_NOT == u))
agregado$idade_media <- sapply(ufs, function(u)
  mean(dados_brutos$idade_anos_corrigida[dados_brutos$SG_UF_NOT == u], na.rm = TRUE))
agregado$obitos <- sapply(ufs, function(u)
  sum(dados_brutos$EVOLUCAO[dados_brutos$SG_UF_NOT == u] == "2", na.rm = TRUE))
agregado$desfecho_conhecido <- sapply(ufs, function(u)
  sum(dados_brutos$EVOLUCAO[dados_brutos$SG_UF_NOT == u] %in% c("1", "2", "3"), na.rm = TRUE))
agregado$taxa_letalidade <- agregado$obitos / agregado$desfecho_conhecido
```

5 Agregação de todas as proporções (fatores de risco, vacina, UTI)

Mesmas 16 variáveis de 2024, mesmo tratamento especial de `prop_suport_ven` (válido = 1/2/3, “Sim” = 1 ou 2) e mesma exclusão de PUERPERA (só se aplica a mulheres, enviesaria a comparação entre UFs com composição de sexo diferente entre os casos).

```
mapa_variaveis <- c(
  CARDIOPATI = "prop_cardiopatia", DIABETES = "prop_diabetes",
  OBESIDADE = "prop_obesidade", VACINA = "prop_vacina_gripe",
  VACINA_COV = "prop_vacina_covid", UTI = "prop_uti",
  ASMA = "prop_asma", RENAL = "prop_renal",
  IMUNODEPRE = "prop_imunodepre", NEUROLOGIC = "prop_neurologic",
  PNEUMOPATI = "prop_pneumopati", TABAG = "prop_tabag",
  HEPATICA = "prop_hepatica", HEMATOLOGI = "prop_hematologi",
  SIND_DOWN = "prop_sind_down"
)

for (campo in names(mapa_variaveis)) {
  nome_col <- mapa_variaveis[[campo]]
  agregado[[nome_col]] <- sapply(ufs, function(u)
    prop_sim(dados_brutos[[campo]][dados_brutos$SG_UF_NOT == u]))
}

agregado$prop_suport_ven <- sapply(ufs, function(u) {
  x <- as.character(dados_brutos$SUPORT_VEN[dados_brutos$SG_UF_NOT == u])
  denom <- sum(x %in% c("1", "2", "3"), na.rm = TRUE)
  if (denom == 0) return(NA)
  sum(x %in% c("1", "2"), na.rm = TRUE) / denom
})

# [PREENCHER] TABAG não tinha nenhuma resposta válida em 2024 - confirme se
# ainda é o caso em 2026 (se sim, prop_tabag fica só NA e sai das candidatas)
sum(!is.na(dados_brutos$TABAG))
```

```
## [1] 0
```

6 Conferência

```
nrow(agregado) # deve ser 27
```

```
## [1] 27
```

```
colSums(is.na(agregado))
```

```
##           UF           n_casos      idade_media      obitos
##           0             0         0             0
## desfecho_conhecido taxa_letalidade prop_cardiopatias prop_diabetes
##           0             0         0             0
##      prop_obesidade prop_vacina_gripe prop_vacina_covid      prop_uti
##           0             0         0             0
##      prop_asma      prop_renal      prop_imunodepre prop_neurológic
##           0             0         0             0
##      prop_pneumopati      prop_tabag      prop_hepática prop_hematologi
##           0             27         0             0
##      prop_sind_down      prop_suport_ven
##           0             0
```

```
agregado
```

```
##   UF n_casos idade_media obitos desfecho_conhecido taxa_letalidade
## 1  RO   1122   18.42380    26         644      0.040372671
## 2  RJ   13520  18.52463   656        11365     0.057721073
## 3  DF    6780  11.84387    52         6162     0.008438819
## 4  MS    5808  25.99586   425         5172     0.082173241
## 5  PA    4727  22.89070   197         3606     0.054631170
## 6  GO    5763  25.82229   420         5258     0.079878281
## 7  AM    3468  26.66181    85         2415     0.035196687
## 8  SP   44497  26.17718  2055        37455     0.054865839
## 9  BA    7643  14.43403   234         6549     0.035730646
## 10 PR   18918  32.90630   863        15667     0.055083934
## 11 PE    5015  14.92434    80         2929     0.027313076
## 12 CE    7538  23.44714   329         6027     0.054587689
## 13 MA    2576  20.16373   155         2048     0.075683594
## 14 SE    2378  11.86650    79         1895     0.041688654
## 15 PB    3963  20.74337   297         3406     0.087199060
## 16 AC    2346  22.74067    62         1791     0.034617532
## 17 AP    1742  13.55939    31         1620     0.019135802
## 18 MG   26412  32.06327  1218        22620     0.053846154
## 19 RR    1227  12.51731    44         1064     0.041353383
## 20 TO     680  15.84935    30         446      0.067264574
## 21 MT    3851  20.68423   147         3048     0.048228346
## 22 AL    1657  19.86699    71         807      0.087980173
## 23 RS   11976  33.72426   781         9311     0.083879283
```

## 24	ES	1683	25.08284	78	1477	0.052809749
## 25	SC	10231	24.22651	434	9056	0.047924028
## 26	RN	1578	17.20192	71	1181	0.060118544
## 27	PI	1004	38.60379	102	669	0.152466368
##	prop_cardiopatia	prop_diabetes	prop_obesidade	prop_vacina_gripe		
## 1	0.3944444	0.2808989	0.12030075	0.2277040		
## 2	0.4630493	0.2502740	0.05915268	0.1268484		
## 3	0.3218659	0.1792976	0.04265092	0.1570561		
## 4	0.3854875	0.2318501	0.05623472	0.2021982		
## 5	0.3838951	0.2442748	0.04429134	0.2749591		
## 6	0.3804406	0.2326218	0.04088586	0.1643400		
## 7	0.4212963	0.4093098	0.05415861	0.3204294		
## 8	0.4265022	0.2776339	0.06315379	0.2076671		
## 9	0.3185680	0.2316919	0.05942623	0.1509046		
## 10	0.4086604	0.2595398	0.07934132	0.2733102		
## 11	0.3518851	0.2443182	0.05679513	0.1263236		
## 12	0.3680033	0.2797858	0.07684478	0.1549872		
## 13	0.4670051	0.4278351	0.07453416	0.1129758		
## 14	0.4089835	0.2135922	0.07258065	0.1615721		
## 15	0.4807520	0.2790698	0.05968779	0.1508133		
## 16	0.2648649	0.2323232	0.05504587	0.1594533		
## 17	0.2961039	0.2052632	0.03252033	0.2558140		
## 18	0.4063902	0.2506530	0.07419108	0.2322641		
## 19	0.4421053	0.2540984	0.05164319	0.1959391		
## 20	0.5000000	0.3000000	0.04597701	0.3011364		
## 21	0.3875339	0.2917232	0.06295754	0.1158093		
## 22	0.4598765	0.3761755	0.06766917	0.1781502		
## 23	0.4447332	0.2715149	0.07700914	0.2840391		
## 24	0.5465950	0.3140496	0.07674419	0.1766504		
## 25	0.4315903	0.2584270	0.07438017	0.2347946		
## 26	0.2832765	0.1879310	0.06360424	0.1535459		
## 27	0.6987952	0.4378531	0.07508532	0.1716102		
##	prop_vacina_covid	prop_uti	prop_asma	prop_renal	prop_imunodepre	
## 1	0.2529148	0.2963526	0.4082840	0.12857143	0.04580153	
## 2	0.3261181	0.3500728	0.3171589	0.09283196	0.09638095	
## 3	0.2356773	0.2974598	0.4319863	0.06282383	0.10264290	
## 4	0.3858771	0.1897585	0.1894484	0.05905320	0.06508014	
## 5	0.3582154	0.1762546	0.1864734	0.08406647	0.09264565	
## 6	0.4063103	0.3197533	0.1633880	0.06815642	0.06350975	
## 7	0.4339843	0.1994721	0.3355049	0.11111111	0.07536765	
## 8	0.4255295	0.2874860	0.2571758	0.06929254	0.07380862	
## 9	0.2921767	0.4352068	0.3626698	0.07915567	0.08741031	
## 10	0.5126481	0.2279855	0.2079558	0.07671074	0.05613577	
## 11	0.2457059	0.2746337	0.3072917	0.06336634	0.08464567	
## 12	0.4160249	0.2521229	0.2597712	0.06865672	0.11455261	
## 13	0.3137100	0.3284899	0.1793478	0.11176471	0.08011869	
## 14	0.2754491	0.3483084	0.3051643	0.05774278	0.08776596	
## 15	0.3330757	0.3025015	0.1909414	0.06005460	0.05439710	
## 16	0.3543032	0.1685626	0.2659574	0.09411765	0.19827586	
## 17	0.2698504	0.1653778	0.2605263	0.04812834	0.01876676	
## 18	0.4899566	0.2206553	0.2070973	0.07860345	0.08204835	
## 19	0.2223206	0.2776860	0.2694064	0.10714286	0.09292035	
## 20	0.2567976	0.2135922	0.1300000	0.12903226	0.14285714	
## 21	0.3083551	0.2700908	0.1069364	0.11046512	0.07837446	

## 22	0.3177115	0.4814159	0.2474227	0.03533569	0.12758621
## 23	0.4624025	0.2802683	0.2232516	0.06523765	0.07357357
## 24	0.3731254	0.5279421	0.3757116	0.08783784	0.07207207
## 25	0.3405427	0.2815922	0.2686883	0.05706626	0.09670330
## 26	0.2684650	0.3478557	0.2847222	0.06964286	0.07746479
## 27	0.6120163	0.3179191	0.2369231	0.09935897	0.08813559
##	prop_neurologic	prop_pneumopati	prop_tabag	prop_hepatica	prop_hematologi
## 1	0.36942675	0.12765957	NA	0.028368794	0.049645390
## 2	0.18382628	0.16610676	NA	0.023218575	0.038643533
## 3	0.16493586	0.18502475	NA	0.012250161	0.044614404
## 4	0.12312892	0.13085005	NA	0.011713031	0.016152717
## 5	0.13967022	0.10731707	NA	0.009852217	0.008928571
## 6	0.13211719	0.11812738	NA	0.012055109	0.017947280
## 7	0.16062176	0.24564460	NA	0.017110266	0.005825243
## 8	0.13855059	0.13564268	NA	0.016561393	0.034963963
## 9	0.17150063	0.09716860	NA	0.008626410	0.050032916
## 10	0.14236361	0.18411706	NA	0.014072120	0.022258017
## 11	0.12713472	0.11641221	NA	0.017964072	0.019920319
## 12	0.26026743	0.13285372	NA	0.020897044	0.037525355
## 13	0.11830986	0.06231454	NA	0.008955224	0.020588235
## 14	0.18427518	0.10406091	NA	0.018518519	0.020779221
## 15	0.18705036	0.12375790	NA	0.011882998	0.021978022
## 16	0.10704225	0.08656716	NA	0.060518732	0.014749263
## 17	0.12032086	0.05851064	NA	0.005333333	0.005347594
## 18	0.12556861	0.22227439	NA	0.017513638	0.028770790
## 19	0.16129032	0.21991701	NA	0.028846154	0.028037383
## 20	0.09574468	0.12903226	NA	0.032608696	0.031914894
## 21	0.15706052	0.23110465	NA	0.030612245	0.035346097
## 22	0.11986301	0.10689655	NA	0.038461538	0.028070175
## 23	0.14531655	0.21659660	NA	0.016150235	0.020712820
## 24	0.17291667	0.20208333	NA	0.013761468	0.025287356
## 25	0.12058603	0.19207994	NA	0.012648524	0.034690799
## 26	0.18101933	0.07381371	NA	0.014260250	0.012411348
## 27	0.17295597	0.13680782	NA	0.010600707	0.017182131
##	prop_sind_down	prop_suport_ven			
## 1	0.03821656	0.6713819			
## 2	0.03497615	0.5982510			
## 3	0.04891995	0.7369359			
## 4	0.02830649	0.7925319			
## 5	0.02270484	0.5128582			
## 6	0.01897321	0.7516438			
## 7	0.04469274	0.6608564			
## 8	0.02144115	0.6408239			
## 9	0.03963613	0.4820235			
## 10	0.02177089	0.5948354			
## 11	0.05078125	0.5248958			
## 12	0.03881048	0.5893189			
## 13	0.02906977	0.5273743			
## 14	0.01570681	0.6217617			
## 15	0.01634877	0.4908913			
## 16	0.01759531	0.4101280			
## 17	0.03733333	0.3945978			
## 18	0.02186993	0.6278690			
## 19	0.03773585	0.7080901			


```
## 20      0.03191489      0.4543081
## 21      0.02496329      0.3683856
## 22      0.02739726      0.6804933
## 23      0.02550261      0.7862767
## 24      0.02288330      0.7890354
## 25      0.02819048      0.6408846
## 26      0.02120141      0.6772697
## 27      0.02960526      0.6697248
```

7 Passo 1 — Correlação de cada candidata com a resposta

Mesmo raciocínio de 2024: para regressão simples ($k = 1$), $R^2 = r^2$, então o ranking por correlação já indica quem produziria o maior ajuste individual. Isso deixa de valer a partir de dois regressores (Passos 2–3), por isso é só o ponto de partida, não o critério final.

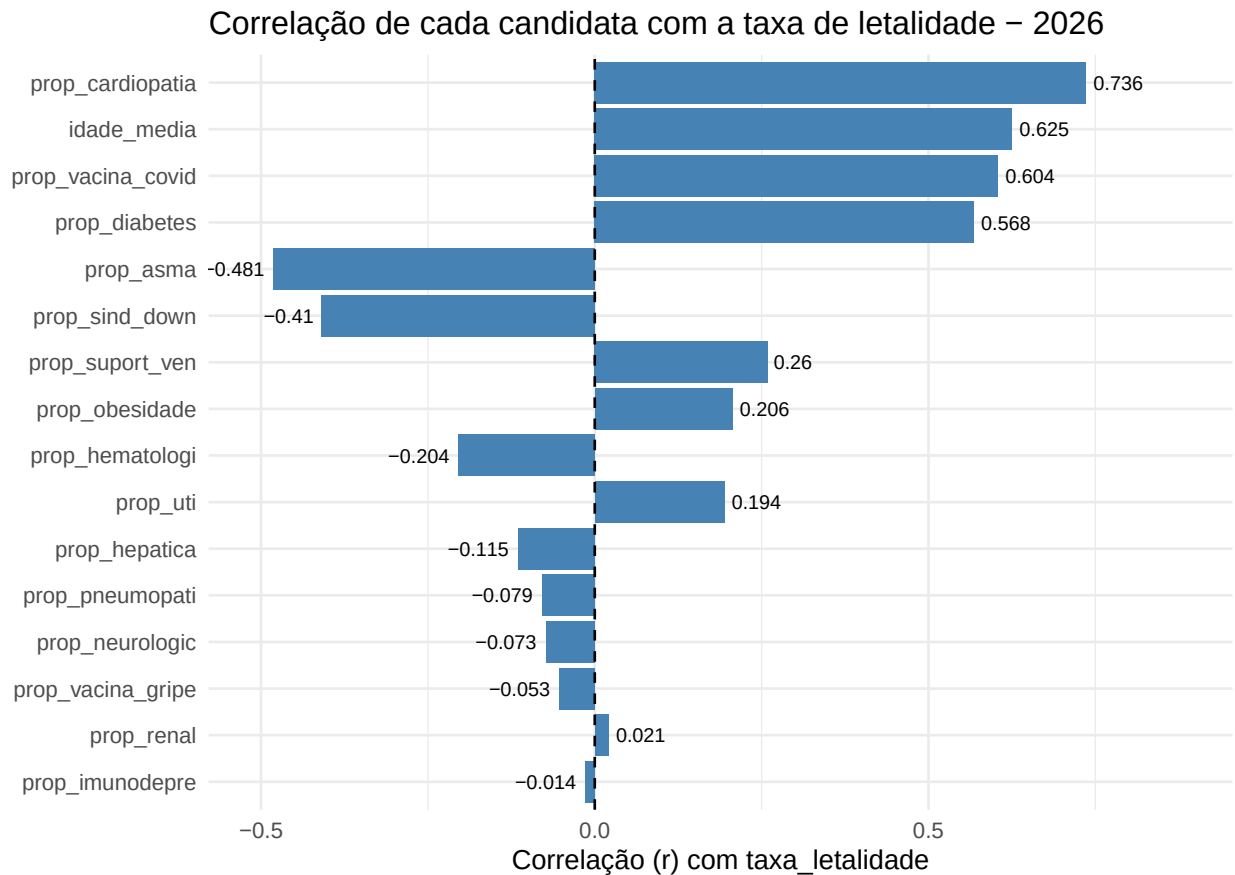
```
candidatas <- c("idade_media", "prop_cardiopatia", "prop_diabetes", "prop_obesidade",
               "prop_vacina_gripe", "prop_vacina_covid", "prop_uti", "prop_asma",
               "prop_renal", "prop_imunodepre", "prop_neurologic", "prop_pneumopati",
               "prop_hepatica", "prop_hematologi", "prop_sind_down",
               "prop_suport_ven")
# remove candidatas sem variação/dados suficientes (ex.: prop_tabag em 2024)
candidatas <- candidatas[sapply(candidatas, function(v) sum(!is.na(agregado[[v]])) >= 10)]

cor_y <- sapply(candidatas, function(v) cor(agregado[[v]], agregado$taxa_letalidade, use = "complete.obs"))
df_cor <- data.frame(variavel = names(cor_y), correlacao = round(cor_y, 4))
df_cor <- df_cor[order(-abs(df_cor$correlacao)), ]
row.names(df_cor) <- NULL
df_cor
```

```
##          variavel correlacao
## 1  prop_cardiopatia    0.7363
## 2      idade_media    0.6252
## 3 prop_vacina_covid    0.6041
## 4      prop_diabetes    0.5679
## 5      prop_asma     -0.4811
## 6 prop_sind_down     -0.4097
## 7 prop_suport_ven    0.2595
## 8      prop_obesidade    0.2062
## 9 prop_hematologi   -0.2041
## 10      prop_uti     0.1941
## 11 prop_hepatica    -0.1149
## 12 prop_pneumopati   -0.0791
## 13 prop_neurologic   -0.0734
## 14 prop_vacina_gripe  -0.0531
## 15      prop_renal    0.0207
## 16 prop_imunodepre   -0.0142
```

```
ggplot(df_cor, aes(x = correlacao, y = reorder(variavel, abs(correlacao)))) +
  geom_col(fill = "steelblue") +
  geom_vline(xintercept = 0, linetype = "dashed") +
  geom_text(aes(label = round(correlacao, 3)),
            hjust = ifelse(df_cor$correlacao >= 0, -0.15, 1.15),
```

```
size = 2.8) +
scale_x_continuous(expand = expansion(mult = c(0.08, 0.18))) +
labs(title = paste("Correlação de cada candidata com a taxa de letalidade -", ANO_ANALISE_REG),
x = "Correlação (r) com taxa_letalidade", y = NULL) +
theme_minimal() +
theme(axis.text.y = element_text(size = 9))
```



[PREENCHER] Compare este ranking com o de 2024 (lá: idade_media 0,713 > prop_vacina_covid 0,622 > prop_sind_down -0,593 > prop_diabetes 0,591 > prop_asma -0,516 > prop_uti 0,513 > prop_cardiopatia 0,501). As mesmas variáveis lideram em 2026?

8 Passo 2 — Modelo preliminar com as 5 maiores correlações entre as candidatas clinicamente diretas (1ª rodada)

Mesmo recorte de 2024: das candidatas com relevância clínica mais óbvia para gravidade de SRAG (idade, UTI, cardiopatia, diabetes, obesidade e as duas vacinas), ficamos com as **5 de maior |r|** — não fixamos quais são de antemão, o código escolhe a partir do ranking de 2026.

```
candidatas_principais <- intersect(
  c("idade_media", "prop_uti", "prop_cardiopatia", "prop_diabetes",
    "prop_obesidade", "prop_vacina_gripe", "prop_vacina_covid"),
  candidatas
```

```
)
ranking_principais <- df_cor[df_cor$variavel %in% candidatas_principais, ]
ranking_principais
```

```
##          variavel correlacao
## 1  prop_cardiopatia    0.7363
## 2      idade_media    0.6252
## 3 prop_vacina_covid    0.6041
## 4      prop_diabetes    0.5679
## 8      prop_obesidade    0.2062
## 10      prop_uti        0.1941
## 14 prop_vacina_gripe   -0.0531
```

```
top5_variaveis <- head(ranking_principais$variavel, 5)
top5_variaveis
```

```
## [1] "prop_cardiopatia" "idade_media"      "prop_vacina_covid"
## [4] "prop_diabetes"     "prop_obesidade"
```

```
formula_top5 <- reformulate(top5_variaveis, response = "taxa_letalidade")
formula_top5
```

```
## taxa_letalidade ~ prop_cardiopatia + idade_media + prop_vacina_covid +
##      prop_diabetes + prop_obesidade
```

```
modelo_top5 <- lm(formula_top5, data = agregado)
summary(modelo_top5)
```

```
##
## Call:
## lm(formula = formula_top5, data = agregado)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.032990 -0.010066 -0.001542  0.009218  0.033645
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -0.040834   0.022743  -1.795  0.08699 .
## prop_cardiopatia  0.180052   0.061534   2.926  0.00807 **
## idade_media      0.001976   0.001895   1.043  0.30898
## prop_vacina_covid -0.038456   0.137648  -0.279  0.78268
## prop_diabetes     0.013825   0.078555   0.176  0.86199
## prop_obesidade   -0.156492   0.238944  -0.655  0.51962
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01851 on 21 degrees of freedom
## Multiple R-squared:  0.6472, Adjusted R-squared:  0.5632
## F-statistic: 7.706 on 5 and 21 DF, p-value: 0.0002999
```

[PREENCHER] Olhe o `summary()` acima: o teste F do modelo é significativo mas os coeficientes individuais não são (repete o sintoma de multicolinearidade de 2024)? Ou já sai diferente desta vez?

9 Passo 3 — Diagnóstico de multicolinearidade (VIF e correlação entre regressores)

$$VIF_j = \frac{1}{1 - R_j^2}$$

Critério prático: VIF > 10 indica multicolinearidade preocupante; alguns autores usam o limiar mais conservador de 5.

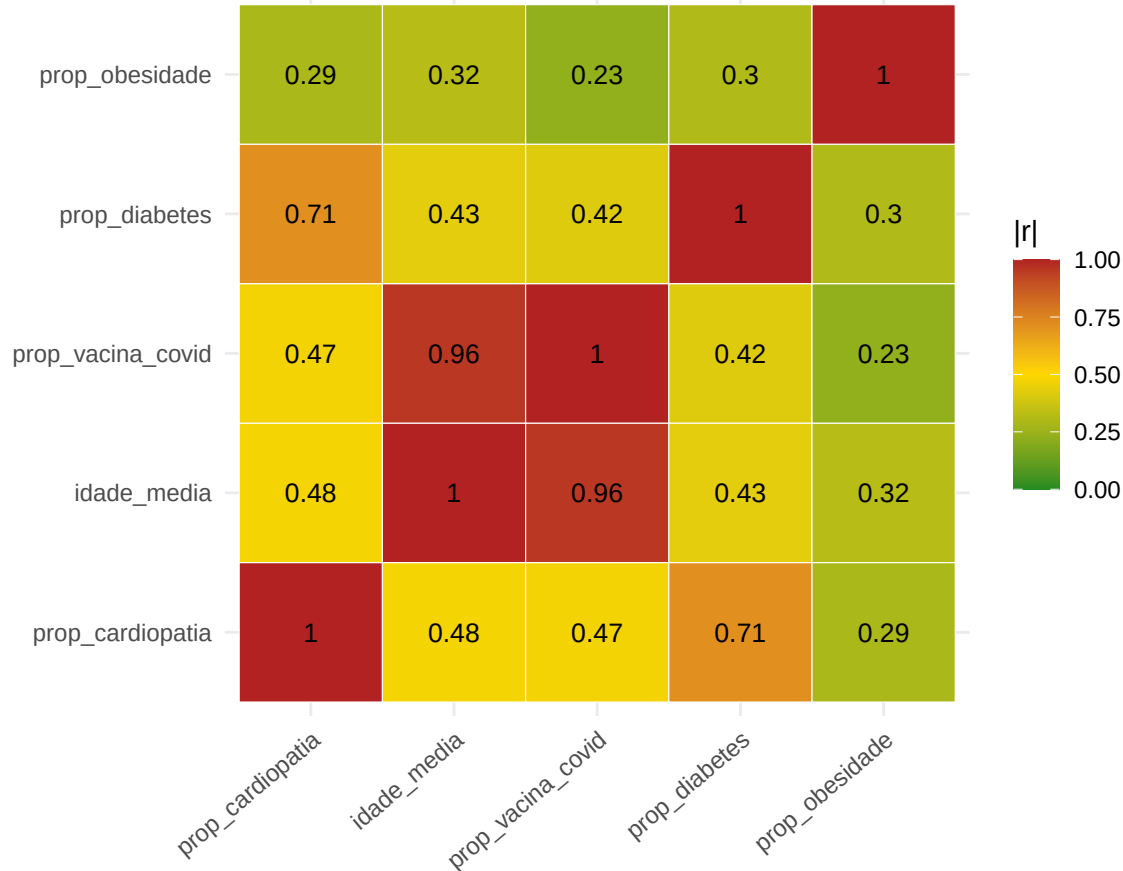
```
vif(modelo_top5)
```

```
## prop_cardiopatias idade_media prop_vacina_covid prop_diabetes
##      2.220480      13.538883      12.987801      2.115449
## prop_obesidade
##      1.271708
```

```
matriz_top5 <- cor(agregado[, top5_variaveis], use = "complete.obs")
matriz_long <- as.data.frame(as.table(matriz_top5))
names(matriz_long) <- c("var1", "var2", "correlacao")
matriz_long$abs_correlacao <- abs(matriz_long$correlacao)

ggplot(matriz_long, aes(x = var1, y = var2, fill = abs_correlacao)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(correlacao, 2)), size = 3.5) +
  scale_fill_gradientn(colors = c("forestgreen", "gold", "firebrick"),
    limits = c(0, 1), name = "|r|") +
  labs(title = paste("Correlação entre as 5 variáveis do modelo preliminar -", ANO_ANALISE_REG),
    x = NULL, y = NULL) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 40, hjust = 1))
```

Correlação entre as 5 variáveis do modelo preliminar – 2026



A regra usada em 2024, agora automatizada em vez de aplicada à mão: entre cada par de regressoras do top5 com $|r| > 0,7$ (redundantes), mantém a que tem maior $|correlação|$ individual com Y (Passo 1) e remove a outra — repete até não sobrar par acima do limiar. É a mesma regra que o slide da disciplina chama de “reespecificação do modelo (eliminação de variáveis)”; a função só evita que a decisão fique implícita (ou seja esquecida) numa variável que precisa ser editada à mão.

```
# Remove, uma de cada vez, a variável de menor |correlação| com Y dentro de
# cada par redundante ( $|r| > \text{limiar}$ ), até não sobrar nenhum par acima do
# limiar. Retorna o subconjunto de `variaveis` sem redundância.
remover_redundantes <- function(variaveis, dados, ranking_cor, limiar = 0.7) {
  repeat {
    if (length(variaveis) < 2) break
    mat <- cor(dados[, variaveis, drop = FALSE], use = "complete.obs")
    diag(mat) <- 0
    if (max(abs(mat)) <= limiar) break
    idx <- which(abs(mat) == max(abs(mat)), arr.ind = TRUE)[1, ]
    v1 <- rownames(mat)[idx[1]]; v2 <- colnames(mat)[idx[2]]
    cor_v1 <- ranking_cor$correlacao[ranking_cor$variavel == v1]
    cor_v2 <- ranking_cor$correlacao[ranking_cor$variavel == v2]
    remover <- if (abs(cor_v1) >= abs(cor_v2)) v2 else v1
    mantido <- setdiff(c(v1, v2), remover)
    message(sprintf(
      "Reespecificação: '%s' removida por redundância com '%s' (r=%.3f) - mantida a de maior |correlação|
    ))
  }
}
```

```

    remover, mantido, mat[idx[1], idx[2]]
  ))
  variaveis <- setdiff(variaveis, remover)
}
variaveis
}

```

10 Passo 4 — Reespecificação: removendo a redundância

```

variaveis_intermediario <- remover_redundantes(top5_variaveis, agregado, df_cor, limiar = 0.7)
variaveis_intermediario

```

```
## [1] "prop_cardiopatia" "idade_media"      "prop_obesidade"
```

```

formula_intermediario <- reformulate(variaveis_intermediario, response = "taxa_letalidade")
modelo_intermediario <- lm(formula_intermediario, data = agregado)
summary(modelo_intermediario)

```

```

##
## Call:
## lm(formula = formula_intermediario, data = agregado)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.03206 -0.01009 -0.00065  0.00962  0.03402
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -0.0435519   0.0188174   -2.314 0.029922 *
## prop_cardiopatia  0.1852707   0.0456050    4.063 0.000482 ***
## idade_media      0.0014866   0.0005743    2.588 0.016434 *
## prop_obesidade  -0.1321104   0.2170867   -0.609 0.548779
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01773 on 23 degrees of freedom
## Multiple R-squared:  0.6454, Adjusted R-squared:  0.5992
## F-statistic: 13.95 on 3 and 23 DF,  p-value: 2.151e-05

```

```
vif(modelo_intermediario)
```

```

## prop_cardiopatia      idade_media      prop_obesidade
##           1.328985           1.355443           1.143771

```

[PREENCHER] Compare com 2024 (lá sobrou `idade_media` + `prop_diabetes` + `prop_uti`). Os VIFs acima caíram pra valores baixos? O R^2 ajustado melhorou ou piorou em relação ao modelo top5 (mensagens acima mostram quais variáveis saíram e por quê)?

11 Passo 5 — Triagem das comorbidades adicionais (2ª rodada)

Mesmo teste de soma de quadrados extra de 2024: para cada candidata que não entrou no recorte “clínicamente direto” do Passo 2, testa se ela contribui significativamente **além** do que o modelo intermediário já explica.

$$F_0 = \frac{\text{SQR}(\beta_{nova} | \dots)/1}{\text{QME}(\text{modelo com a nova variável})} \sim F_{1, n-p}$$

```
candidatas_extra <- setdiff(candidatas, c(candidatas_principais))

resultado_triagem <- data.frame(variavel = candidatas_extra, F_parcial = NA_real_, p_valor = NA_real_)
for (v in candidatas_extra) {
  f <- as.formula(paste(
    "taxa_letalidade ~", paste(variaveis_intermediario, collapse = " + "), "+", v
  ))
  m <- lm(f, data = agregado)
  a <- anova(modelo_intermediario, m)
  resultado_triagem[resultado_triagem$variavel == v, "F_parcial"] <- a$F[2]
  resultado_triagem[resultado_triagem$variavel == v, "p_valor"] <- a$`Pr(>F)`[2]
}
resultado_triagem[order(resultado_triagem$p_valor), ]
```

```
##          variavel      F_parcial      p_valor
## 5 prop_pneumopati 1.677078e+01 0.0004781447
## 1      prop_asma 5.646718e+00 0.0266169872
## 8 prop_sind_down 3.262056e+00 0.0846013692
## 2      prop_renal 1.267495e+00 0.2723693455
## 7 prop_hematologi 6.310393e-01 0.4354602918
## 3 prop_imunodepre 1.308686e-01 0.7209845403
## 6 prop_hepatica 4.652746e-02 0.8312071510
## 4 prop_neurologic 1.311528e-02 0.9098628219
## 9 prop_suport_ven 1.938137e-09 0.9999652705
```

Mesma situação de 2024 pode se repetir: mais de uma candidata passa no teste com $p < 0,05$ mas correlacionada entre si (era o caso de `prop_hematologi` e `prop_sind_down`). A regra do curso é a mesma do Passo 4 — entre um par redundante ($|r| > 0,7$), mantém a de evidência mais forte, aqui medida pelo p-valor do F parcial em vez da correlação com Y. Automatizado pelo mesmo motivo do Passo 4: decisão precisa ficar no código, não numa variável editada à mão.

```
# Entre as candidatas aprovadas na triagem (p<0.05 no F parcial), resolve
# redundância entre pares com |r|>limiar_cor mantendo a de menor p-valor,
# repetindo até não sobrar par redundante. Se nenhuma candidata passar,
# retorna NA (sinal de que o modelo final é o intermediário, sem adição).
selecionar_variavel_nova <- function(resultado, dados, limiar_p = 0.05, limiar_cor = 0.7) {
  aprovadas <- resultado$variavel[!is.na(resultado$p_valor) & resultado$p_valor < limiar_p]
  if (length(aprovadas) == 0) return(NA_character_)
  repeat {
    if (length(aprovadas) < 2) break
    mat <- cor(dados[, aprovadas, drop = FALSE], use = "complete.obs")
    diag(mat) <- 0
    if (max(abs(mat)) <= limiar_cor) break
  }
}
```

```

idx <- which(abs(mat) == max(abs(mat)), arr.ind = TRUE)[1, ]
v1 <- rownames(mat)[idx[1]]; v2 <- colnames(mat)[idx[2]]
p_v1 <- resultado$p_valor[resultado$variavel == v1]
p_v2 <- resultado$p_valor[resultado$variavel == v2]
remover <- if (p_v1 <= p_v2) v2 else v1
mantido <- setdiff(c(v1, v2), remover)
message(sprintf(
  "Triagem: '%s' removida por redundância com '%s' (r=%.3f) - mantida a de menor p-valor no F parcial",
  remover, mantido, mat[idx[1], idx[2]]
))
aprovadas <- setdiff(aprovadas, remover)
}
aprovadas[which.min(resultado$p_valor[resultado$variavel %in% aprovadas])]
}

variavel_nova <- selecionar_variavel_nova(resultado_triagem, agregado)
if (is.na(variavel_nova)) {
  message("Nenhuma candidata passou na triagem (p<0,05) - modelo final = modelo intermediário do Passo 4")
} else {
  message("Variável escolhida na triagem: ", variavel_nova)
}
variavel_nova

```

```
## [1] "prop_pneumopati"
```

12 Passo 6 — a nova variável torna alguma do modelo intermediário desnecessária?

Mesma lógica de 2024 (lá, `prop_hematologi` tornou `prop_diabetes` dispensável): entra a `variavel_nova` escolhida no Passo 5, olha se alguma variável do intermediário perdeu significância individual ($p > 0,05$) e, se sim, testa removê-la comparando $R^2_{ajustado}$ e AIC (critério de 2024: $R^2_{ajustado}$ maior e/ou AIC menor = melhor). Repete até sobrar só variáveis que, ou são significativas, ou cuja remoção não melhora o modelo. A `variavel_nova` em si nunca é candidata a sair aqui — ela é o que está sendo testado.

```

if (!is.na(variavel_nova)) {
  formula_final_teste <- reformulate(c(variaveis_intermediario, variavel_nova), response = "taxa_letalidade")
  modelo_teste <- lm(formula_final_teste, data = agregado)
  summary(modelo_teste)$coefficients
} else {
  message("Sem variável nova - nada a testar aqui; modelo final = modelo intermediário do Passo 4.")
}

```

```
##               Estimate Std. Error  t value    Pr(>|t|)
## (Intercept) -0.030700852 0.0148292282 -2.0702933 5.036346e-02
## prop_cardiopatia 0.203871812 0.0354181167  5.7561449 8.632525e-06
## idade_media  0.001966336 0.0004576178  4.2968957 2.921573e-04
## prop_obesidade -0.120686174 0.1672265349 -0.7216927 4.780843e-01
## prop_pneumopati -0.219122080 0.0535068607 -4.0952147 4.781447e-04
```



```

# Elimina, uma de cada vez, variáveis do intermediário que perderam
# significância (p>0.05) e cuja remoção melhora R2 ajustado ou AIC - nunca
# remove `variavel_nova`. Sem `variavel_nova` (Passo 5 não aprovou nada),
# devolve o intermediário sem alteração.
decidir_variaveis_finais <- function(variaveis_base, variavel_nova, dados) {
  if (is.na(variavel_nova)) return(variaveis_base)

  variaveis_atuais <- c(variaveis_base, variavel_nova)
  modelo_atual <- lm(reformulate(variaveis_atuais, response = "taxa_letalidade"), data = dados)

  repeat {
    coefs <- summary(modelo_atual)$coefficients
    candidatas_saida <- setdiff(variaveis_atuais, variavel_nova)
    candidatas_saida <- candidatas_saida[coefs[candidatas_saida, "Pr(>|t|)"] > 0.05]
    if (length(candidatas_saida) == 0) break

    testes <- do.call(rbind, lapply(candidatas_saida, function(v) {
      vars_teste <- setdiff(variaveis_atuais, v)
      m <- lm(reformulate(vars_teste, response = "taxa_letalidade"), data = dados)
      data.frame(variavel = v, R2_ajustado = summary(m)$adj.r.squared, AIC = AIC(m))
    }))

    r2_atual <- summary(modelo_atual)$adj.r.squared
    aic_atual <- AIC(modelo_atual)
    melhor <- testes[order(-testes$R2_ajustado), ][1, ]

    if (melhor$R2_ajustado > r2_atual || melhor$AIC < aic_atual) {
      message(sprintf(
        "Passo 6: '%s' perdeu significância (p>0,05) e sua remoção melhora o ajuste (R2_aj: %.4f -> %.4f)",
        melhor$variavel, r2_atual, melhor$R2_ajustado, aic_atual, melhor$AIC
      ))
      variaveis_atuais <- setdiff(variaveis_atuais, melhor$variavel)
      modelo_atual <- lm(reformulate(variaveis_atuais, response = "taxa_letalidade"), data = dados)
    } else {
      message(sprintf(
        "Passo 6: '%s' perdeu significância individual, mas removê-la não melhora R2 ajustado nem AIC -",
        paste(candidatas_saida, collapse = ", ")
      ))
      break
    }
  }
  variaveis_atuais
}

variaveis_finais <- decidir_variaveis_finais(variaveis_intermediario, variavel_nova, agregado)
variaveis_finais

```

```
## [1] "prop_cardiopatia" "idade_media" "prop_pneumopati"
```

12.1 Resumo do processo de seleção

Os Passos 3-6 agora rodam de ponta a ponta sem edição manual — as mensagens impressas em cada chunk acima (`message()`) documentam cada remoção/adição e por quê. [PREENCHER] só o que é leitura, não

decisão: preencha a tabela abaixo olhando os resultados já produzidos, e comente se o padrão bate com 2024.

Etapa	Ação	Resultado
1	Correlação com Y	[PREENCHER] variáveis no topo do ranking
2	Top 5 das candidatas principais	[PREENCHER] quais 5 entraram
3	VIF do top5	[PREENCHER] houve multicolinearidade severa?
4	Reespecificação	variaveis_intermediario (mensagens acima)
5	Triagem de comorbidades extras	variavel_nova (mensagens acima)
6	Verificação de variável desnecessária	variaveis_finais (mensagens acima)

Modelo final: definido automaticamente por **variaveis_finais** (chunk anterior) — a fórmula abaixo é gerada a partir dele, não digitada à mão.

```
modelo <- lm(reformulate(variaveis_finais, response = "taxa_letalidade"), data = agregado)
summary(modelo)
```

```
##
## Call:
## lm(formula = reformulate(variaveis_finais, response = "taxa_letalidade"),
##     data = agregado)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.024488 -0.008792 -0.002574  0.013209  0.022197
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -0.0351117   0.0133697  -2.626  0.015096 *
## prop_cardiopatia  0.1997515   0.0345889   5.775 6.97e-06 ***
## idade_media      0.0018992   0.0004434   4.284 0.000278 ***
## prop_pneumopati -0.2197663   0.0529392  -4.151 0.000386 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01351 on 23 degrees of freedom
## Multiple R-squared:  0.794, Adjusted R-squared:  0.7672
## F-statistic: 29.56 on 3 and 23 DF, p-value: 4.568e-08
```

13 Ajuste e Diagnóstico do Modelo Final

A partir daqui o código é genérico — funciona com qualquer **modelo** definido acima, sem precisar editar mais nada.

13.1 Tabela ANOVA

```
anova(modelo)

## Analysis of Variance Table
##
## Response: taxa_letalidade
##           Df      Sum Sq   Mean Sq F value    Pr(>F)
## prop_cardiopatia  1 0.0110551 0.0110551   60.531 6.895e-08 ***
## idade_media      1 0.0019908 0.0019908   10.901 0.0031180 **
## prop_pneumopati  1 0.0031474 0.0031474   17.233 0.0003861 ***
## Residuals       23 0.0042006 0.0001826
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

13.2 Intervalos de confiança dos coeficientes

```
confint(modelo, level = 0.95)

##              2.5 %      97.5 %
## (Intercept) -0.0627690252 -0.007454320
## prop_cardiopatia  0.1281988911  0.271304138
## idade_media      0.0009820141  0.002816371
## prop_pneumopati -0.3292793464 -0.110253169
```

13.3 Diagnóstico de multicolinearidade (VIF e índice de condição)

```
vif(modelo)

## prop_cardiopatia      idade_media prop_pneumopati
##           1.316098           1.390550           1.166070

X_final <- agregado[, variaveis_finais, drop = FALSE]
autovalores_final <- eigen(cor(X_final, use = "complete.obs"))$values
kappa_final <- max(autovalores_final) / autovalores_final
data.frame(autovalor = round(autovalores_final, 4),
           indice_condicao = round(kappa_final, 2))

##   autovalor indice_condicao
## 1    1.7469           1.00
## 2    0.7400           2.36
## 3    0.5131           3.40
```

13.4 Comparação com o modelo reduzido (teste F parcial da última variável adicionada)

```

# Só faz sentido comparar "com" vs. "sem a última variável adicionada" se
# essa variável (variavel_nova) de fato sobreviveu ao Passo 6 e sobrou mais
# de uma variável no modelo final. Sem essa checagem, quando a triagem não
# aprova nada (variavel_nova = NA) ou o Passo 6 remove a própria
# variavel_nova, "reduzido" e "modelo" ficam idênticos e a comparação abaixo
# zera (Sum of Sq = 0, mesmos R2_ajustado/AIC) - foi exatamente esse o bug
# da primeira versão deste documento, quando essa decisão dependia de editar
# variáveis à mão no meio do Rmd.
if (!is.na(variavel_nova) && variavel_nova %in% variaveis_finais && length(variaveis_finais) > 1) {
  variaveis_reduzido <- setdiff(variaveis_finais, variavel_nova)
  modelo_reduzido <- lm(reformulate(variaveis_reduzido, response = "taxa_letalidade"), data = agregado)

  anova(modelo_reduzido, modelo)

  data.frame(
    modelo = c(paste(variaveis_reduzido, collapse = " + "), paste(variaveis_finais, collapse = " + ")),
    R2_ajustado = c(summary(modelo_reduzido)$adj.r.squared, summary(modelo)$adj.r.squared),
    AIC = c(AIC(modelo_reduzido), AIC(modelo))
  )
} else {
  message(
    "Não há teste F parcial a fazer aqui: nenhuma variável da triagem (Passo 5) ",
    "entrou no modelo final, ou ela foi removida no Passo 6. O modelo final é ",
    "igual ao modelo intermediário do Passo 4."
  )
}

```

```

##                                modelo R2_ajustado      AIC
## 1                prop_cardiopatia + idade_media    0.6096728 -137.0248
## 2 prop_cardiopatia + idade_media + prop_pneumopati  0.7671614 -150.1232

```

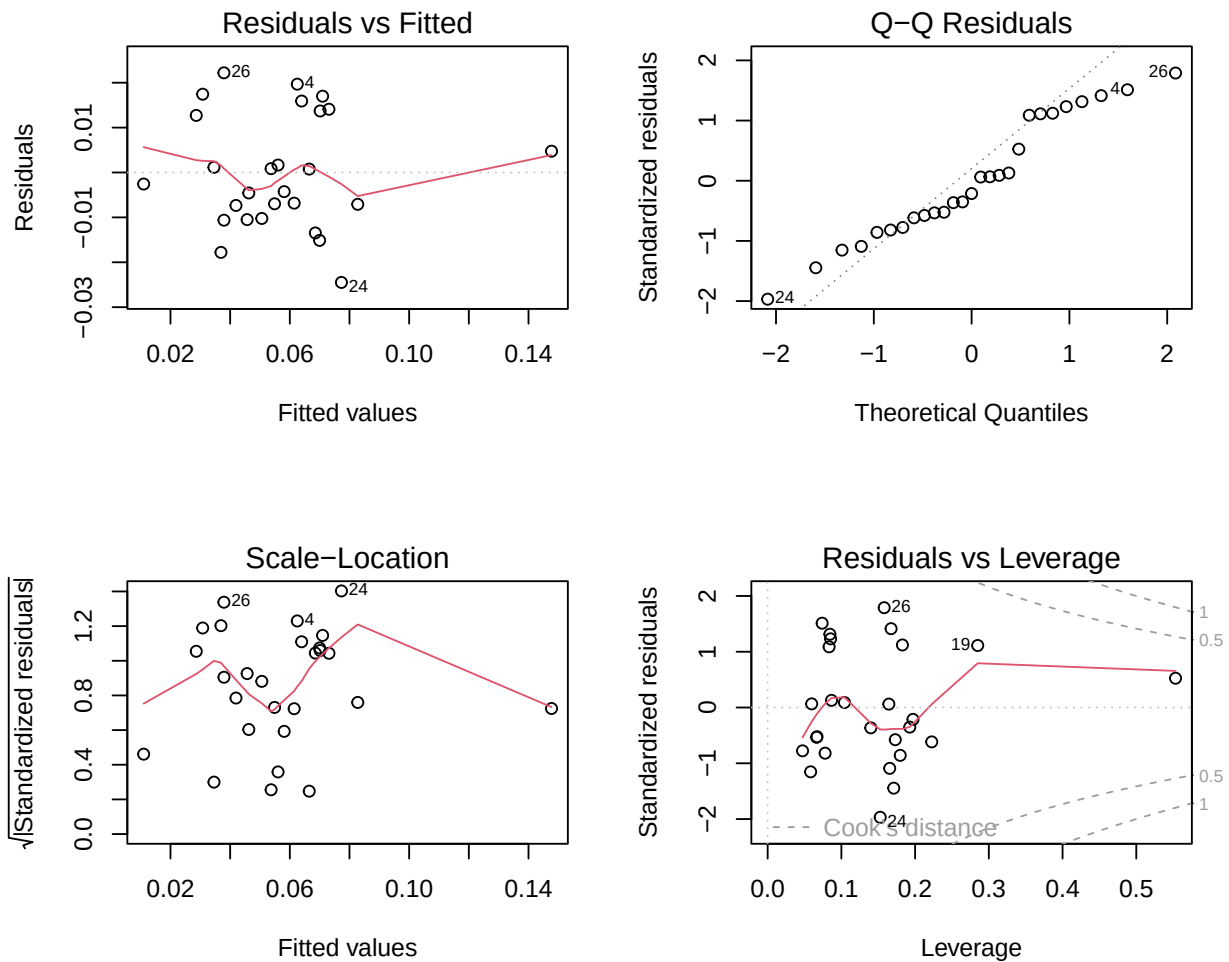
13.5 Gráficos de diagnóstico padrão dos resíduos

- Resíduos vs Ajustados: linearidade.
- Normal Q-Q: normalidade dos erros.
- Scale-Location: homocedasticidade.
- Resíduos vs Alavancagem: pontos potencialmente influentes.

```

par(mfrow = c(2, 2))
plot(modelo)

```



```
par(mfrow = c(1, 1))
```

13.6 Resíduos studentizados internos e R-student

```
residuos_padronizados <- rstandard(modelo)
r_student <- rstudent(modelo)

tabela_residuos <- data.frame(
  UF = agregado$UF,
  residuo_padronizado = round(residuos_padronizados, 2),
  r_student = round(r_student, 2)
)
tabela_residuos[order(-abs(tabela_residuos$r_student)), ]
```

```
##      UF residuo_padronizado r_student
## 24 ES                -1.97      -2.11
## 26 RN                 1.79       1.89
```

## 4	MS	1.51	1.56
## 17	AP	-1.45	-1.48
## 21	MT	1.41	1.45
## 22	AL	1.31	1.34
## 6	GO	1.23	1.25
## 8	SP	-1.15	-1.16
## 23	RS	1.12	1.13
## 19	RR	1.11	1.12
## 10	PR	-1.09	-1.10
## 15	PB	1.09	1.09
## 7	AM	-0.86	-0.85
## 11	PE	-0.82	-0.81
## 1	RO	-0.78	-0.77
## 16	AC	-0.62	-0.61
## 13	MA	-0.58	-0.57
## 25	SC	-0.53	-0.53
## 27	PI	0.53	0.52
## 5	PA	-0.52	-0.51
## 14	SE	-0.36	-0.36
## 18	MG	-0.35	-0.34
## 3	DF	-0.21	-0.21
## 2	RJ	0.13	0.13
## 9	BA	0.09	0.09
## 12	CE	0.07	0.06
## 20	TO	0.06	0.06

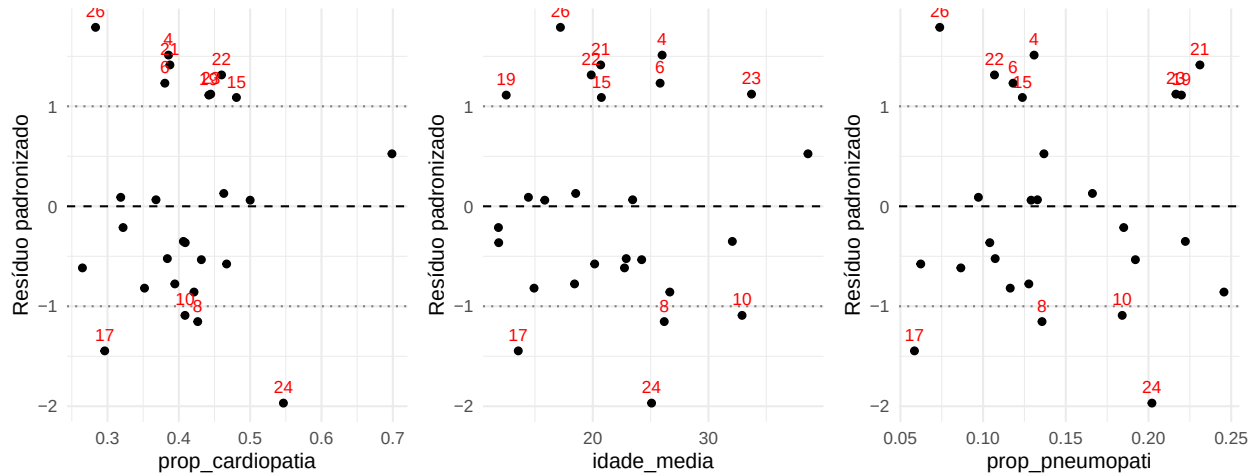
```
# regra prática: |r_student| > 2 sinaliza possível outlier
```

13.7 Resíduos vs. cada regressor

```
agregado$residuo <- residuos_padronizados
agregado$indice <- 1:nrow(agregado)
fora_intervalo <- subset(agregado, abs(residuo) > 1)

graficos_residuo <- lapply(variaveis_finais, function(v) {
  ggplot(agregado, aes(x = .data[[v]], y = residuo)) +
    geom_point() + geom_hline(yintercept = 0, linetype = "dashed") +
    geom_hline(yintercept = c(-1, 1), linetype = "dotted", color = "gray50") +
    geom_text(data = fora_intervalo, aes(label = indice), vjust = -0.8, size = 3, color = "red") +
    labs(x = v, y = "Resíduo padronizado") + theme_minimal()
})

do.call(grid.arrange, c(graficos_residuo, ncol = length(graficos_residuo)))
```



13.8 Estatística PRESS

$$\text{PRESS} = \sum_i \left(\frac{e_i}{1 - h_{ii}} \right)^2$$

Razão PRESS/SQE bem acima de 1 sinaliza sobreajuste.

```
press_residuos <- residuals(modelo) / (1 - lm.influence(modelo)$hat)
PRESS <- sum(press_residuos^2)
SQE <- sum(residuals(modelo)^2)

data.frame(SQE = round(SQE, 5), PRESS = round(PRESS, 5),
           razao_PRESS_SQE = round(PRESS / SQE, 2))
```

```
##      SQE    PRESS razao_PRESS_SQE
## 1 0.0042 0.00575          1.37
```

13.9 Alavancagem e Distância de Cook

$$D_i = \frac{r_i^2}{p} \cdot \frac{h_{ii}}{1 - h_{ii}}$$

Critério prático: $D_i > 4/n$ merece investigação.

```
p <- length(coef(modelo))
n <- nrow(agregado)
limite_hat <- 2 * p / n

hat_values <- lm.influence(modelo)$hat
cooks_d <- cooks.distance(modelo)

tabela_influencia <- data.frame(
  UF = agregado$UF,
  hat = round(hat_values, 3),
  acima_do_limite = hat_values > limite_hat,
```

```

Cooks_D = round(cooks_d, 3)
)
tabela_influencia[order(-tabela_influencia$Cooks_D), ]

```

```

##      UF      hat acima_do_limite Cooks_D
## 24 ES 0.153          FALSE    0.175
## 26 RN 0.158          FALSE    0.150
## 19 RR 0.285          FALSE    0.123
## 17 AP 0.171          FALSE    0.108
## 21 MT 0.167          FALSE    0.101
## 27 PI 0.553           TRUE    0.085
## 23 RS 0.183          FALSE    0.070
## 10 PR 0.166          FALSE    0.059
## 4  MS 0.074          FALSE    0.046
## 7  AM 0.180          FALSE    0.040
## 22 AL 0.084          FALSE    0.040
## 6  GO 0.085          FALSE    0.035
## 15 PB 0.083          FALSE    0.027
## 16 AC 0.223          FALSE    0.027
## 8  SP 0.058          FALSE    0.021
## 13 MA 0.173          FALSE    0.017
## 11 PE 0.078          FALSE    0.014
## 1  RO 0.047          FALSE    0.007
## 18 MG 0.193          FALSE    0.007
## 5  PA 0.067          FALSE    0.005
## 14 SE 0.140          FALSE    0.005
## 25 SC 0.066          FALSE    0.005
## 3  DF 0.197          FALSE    0.003
## 2  RJ 0.087          FALSE    0.000
## 9  BA 0.104          FALSE    0.000
## 12 CE 0.060          FALSE    0.000
## 20 TO 0.164          FALSE    0.000

```

```

limite_hat

```

```

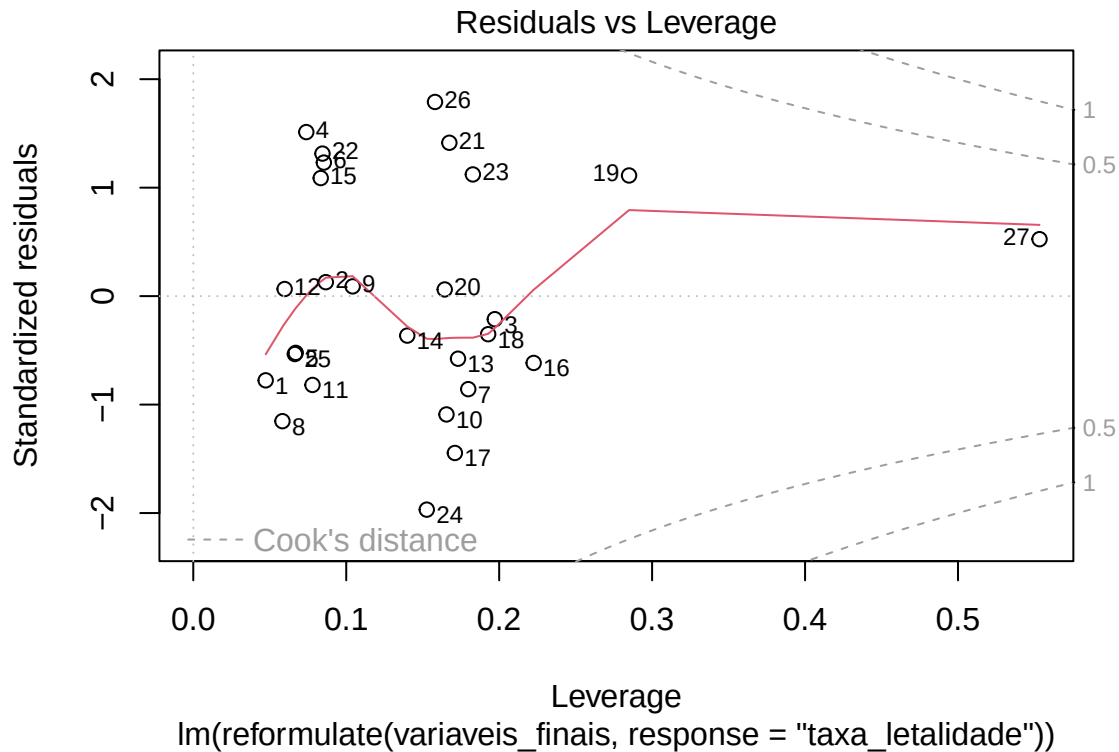
## [1] 0.2962963

```

```

# Em 2024, o Distrito Federal (UF sem municípios, rede hospitalar atípica)
# era o ponto mais influente de longe. [PREENCHER] o mesmo se repete?
plot(modelo, which = 5, id.n = nrow(agregado))

```

14 Equação do Modelo Final

```

coefs <- coef(modelo)
sinal <- function(x) if (x >= 0) "+" else "-"

termos <- paste(sprintf("%s %.4f%s", sapply(names(coefs)[-1], sinal), abs(coefs[-1]), names(coefs)[-1]),
cat(sprintf("taxa_letalidade = %.4f %s\n", coefs["(Intercept)"], termos))

## taxa_letalidade = -0.0351 + 0.1998*prop_cardiopatia + 0.0019*idade_media + 0.2198*prop_pneumopati

cat(sprintf("R2 = %.4f | R2_ajustado = %.4f\n",
summary(modelo)$r.squared, summary(modelo)$adj.r.squared))

## R2 = 0.7940 | R2_ajustado = 0.7672

```

15 Comparação final com 2024

[PREENCHER] — depois de rodar tudo, resuma aqui:

- O modelo final de 2026 usa as mesmas variáveis que 2024 (idade_media + prop_uti + prop_hematologi) ou variáveis diferentes?

- Os sinais dos coeficientes têm a mesma direção (ex.: `prop_hematologi` continua negativa, se ela reaparecer)?
- R^2 ajustado de 2026 é maior, menor ou parecido com o de 2024 (0,761)?
- O Distrito Federal (ou outra UF) continua sendo o ponto de maior influência?